

# Лабораторная работа N3

## Введение

Целью данной работы являлось развитие ранее подготовленного контейнеризированного веб-приложения и его перенос на более полный Kubernetes-стек с постоянным хранением данных. В предыдущей работе было реализовано простое REST-приложение с frontend-страницей, backend-сервисом и развертыванием в Kubernetes с использованием Deployment, ConfigMap, Service и маршрутизации внешнего трафика. В текущей работе приложение было доработано таким образом, чтобы оно сохраняло и отображало данные из базы данных PostgreSQL.

В рамках задания требовалось развернуть PostgreSQL в Kubernetes без использования готовых операторов и Helm-чартов, а именно на основе StatefulSet с использованием Volume и VolumeMounts. Также необходимо было заменить ранее применявшийся Ingress на Gateway API, настроить взаимодействие frontend, backend и базы данных, а затем проверить работоспособность всей системы в кластере.

В качестве прикладного примера было использовано веб-приложение на Python с frontend-частью, отображающей список записей, и backend API, которое обращается к PostgreSQL для чтения и добавления данных. Для изоляции ресурсов был использован отдельный namespace Kubernetes.

Конфигурация приложения и параметры подключения к базе данных были вынесены в ConfigMap и Secret, а маршрутизация пользовательского трафика организована через Gateway и HTTPRoute.

Таким образом, в работе был получен практический опыт:

- доработки приложения для работы с PostgreSQL;
- развертывания PostgreSQL в Kubernetes через StatefulSet;
- настройки постоянного хранилища;
- использования Gateway API вместо Ingress;
- проверки работоспособности распределенного приложения в среде из master- и worker-ноды.

## Подготовка структуры проекта и выбор архитектуры

В качестве основы была использована предыдущая работа, в которой уже было подготовлено контейнеризированное приложение, состоящее из frontend- и backend-частей. В новой работе архитектура была расширена за счет подключения базы данных PostgreSQL и переноса хранения данных из временного состояния приложения в постоянное хранилище.

Проект был организован в виде нескольких частей:

- frontend с HTML- и JavaScript-кодом пользовательского интерфейса;
- backend с Python-приложением, реализующим REST API;
- каталог k8s с Kubernetes-манифестами для namespace, ConfigMap, Deployment, StatefulSet, Service, Gateway и PersistentVolume.

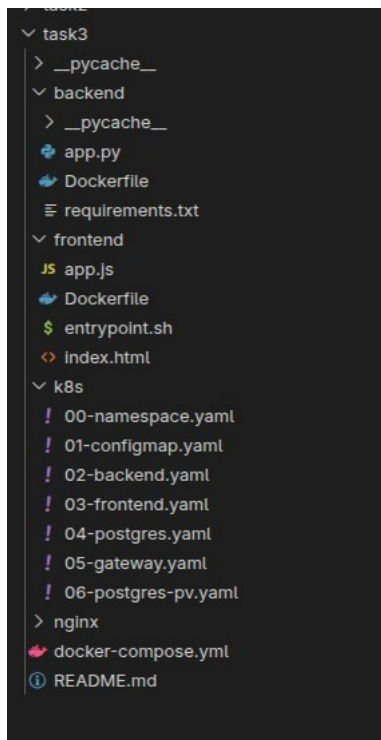
На этом этапе была определена целевая схема взаимодействия компонентов:

- пользователь обращается к приложению через Gateway API;
- frontend получает данные через backend API;
- backend подключается к PostgreSQL по сервисному имени внутри

Kubernetes;

- PostgreSQL хранит данные в постоянном томе.

Структура проекта приведена ниже:



## Доработка backend-приложения для работы с PostgreSQL

Backend-приложение на Python было доработано для подключения к PostgreSQL с использованием библиотеки psycopg. Конфигурация подключения была вынесена в переменные окружения, которые затем передавались из Kubernetes через ConfigMap и Secret.

В приложении была реализована логика:

- открытия подключения к PostgreSQL;
- автоматической инициализации таблицы people;
- чтения списка записей через endpoint GET /api/people;
- добавления новой случайной записи через endpoint POST /api/people/random;
- проверки доступности приложения и базы данных через health, startup и ready.

Таким образом, backend перестал хранить данные только в памяти и начал работать с внешней СУБД, что соответствует требованиям задания.

```
return connect(  
    host=os.getenv("POSTGRES_HOST", "localhost"),  
    port=os.getenv("POSTGRES_PORT", "5432"),  
    dbname=os.getenv("POSTGRES_DB", "task3db"),  
    user=os.getenv("POSTGRES_USER", "task3user"),  
    password=os.getenv("POSTGRES_PASSWORD", "task3password"),  
    connect_timeout=int(os.getenv("POSTGRES_CONNECT_TIMEOUT", "3")),  
    row_factory=dict_row,  
)
```

Подключение к бд в бекенд

```
@app.post("/api/people/random")  
def create_random_person():  
    ensure_database_initialized()
```

```
@app.get("/api/people")  
def list_people():  
    ensure_database_initialized()
```

Endpoint в бекенд

## Подготовка контейнеров приложения

Для frontend и backend были подготовлены отдельные Docker-образы. Backend-образ содержал Python-приложение и зависимости, а frontend-образ использовался для раздачи статических файлов пользовательского интерфейса.

```
[maksimka@archlinux ~]$ docker images | grep task3
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
mozlhlgh/task3-backend:latest
    f38013958ad1      217MB      53.4MB
mozlhlgh/task3-frontend:latest
    da930ead5d67      74.3MB     18.1MB
task3-backend:latest
    159bd8fc28e9      217MB      53.4MB      U
task3-frontend:latest
    2595d9ec27a8      74.3MB     18.1MB      U
[maksimka@archlinux ~]$
```

После подготовки Dockerfile-ов образы были собраны и загружены в Docker Hub, чтобы Kubernetes мог использовать их при разворачивании в кластере.

## Развертывание PostgreSQL в Kubernetes через StatefulSet

Для базы данных PostgreSQL были подготовлены Kubernetes-манифесты, включающие:

- Secret для хранения пароля;
- ConfigMap для имени базы данных и пользователя;
- Service для сетевого доступа к PostgreSQL;
- StatefulSet для запуска PostgreSQL как stateful-компонента;
- PersistentVolume и volumeClaimTemplates для постоянного хранения данных.

Использование StatefulSet было необходимо, так как база данных должна сохранять свое состояние между перезапусками pod. Для хранения данных использовался отдельный том, смонтированный в каталог /var/lib/postgresql/data.

```
kind: StatefulSet
metadata:
  name: postgres
  namespace: task3
```

Манифест StatefulSet для разворачивания PostgreSQL в Kubernetes

```
volumeMounts:
  - name: postgres-data
    mountPath: /var/lib/postgresql/data
readinessProbe:
```

Подключение постоянного хранилища к контейнеру PostgreSQL через volumeMounts

```

volumeClaimTemplates:
- metadata:
  name: postgres-data
  spec:
    storageClassName: ""
    accessModes:
    - ReadWriteOnce
    resources:
      requests:
        storage: 1Gi

```

Описание шаблона PVC для автоматического создания хранилища через volumeClaimTemplates

## Настройка backend и frontend в Kubernetes

Для backend и frontend были подготовлены отдельные Deployment и Service. Backend получал параметры подключения к PostgreSQL из ConfigMap и Secret. Для backend также были настроены startupProbe, readinessProbe и livenessProbe, что позволяло Kubernetes корректно определять состояние приложения. Frontend был развернут как отдельный сервис, который предоставлял HTML-страницу и JavaScript-код для обращения к backend API.

```

env:
- name: POSTGRES_HOST
  valueFrom:
    configMapKeyRef:
      name: task3-app-config
      key: POSTGRES_HOST
- name: POSTGRES_PORT
  valueFrom:
    configMapKeyRef:
      name: task3-app-config
      key: POSTGRES_PORT
- name: POSTGRES_DB
  valueFrom:
    configMapKeyRef:
      name: task3-app-config
      key: POSTGRES_DB
- name: POSTGRES_USER
  valueFrom:
    configMapKeyRef:
      name: task3-app-config
      key: POSTGRES_USER
- name: POSTGRES_PASSWORD
  valueFrom:
    secretKeyRef:
      name: postgres-secret
      key: POSTGRES_PASSWORD
- name: POSTGRES_CONNECT_RETRIES
  value: "2"
- name: POSTGRES_CONNECT_DELAY
  value: "1"

```

```
env:
  - name: API_BASE_URL
    valueFrom:
      configMapKeyRef:
        name: task3-app-config
        key: API_BASE_URL
```

## Использование Gateway API вместо Ingress

Вместо Ingress и Ingress Controller в работе была использована современная модель маршрутизации на основе Gateway API. Для этого были созданы:

- Gateway, принимающий HTTP-трафик;
- HTTPRoute, перенаправляющий запросы /api на backend, а остальные запросы на frontend.

Таким образом, был выполнен переход на Gateway API, как требовалось в задании.

```
metadata:
  name: task3-gateway
  namespace: task3
spec:
  gatewayClassName: nginx
  listeners:
    - name: http
      protocol: HTTP
      port: 80
      allowedRoutes:
        namespaces:
          from: Same
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: task3-route
  namespace: task3
spec:
  parentRefs:
    - name: task3-gateway
  rules:
    - matches:
      - path:
          type: PathPrefix
          value: /api
      backendRefs:
        - name: task3-backend
          port: 8080
    - matches:
      - path:
          type: PathPrefix
          value: /
      backendRefs:
        - name: task3-frontend
          port: 80
```

Манифест Gateway и HTTPRoute для маршрутизации внешнего трафика в Kubernetes

## Развертывание приложения в Kubernetes-кластере

После подготовки манифестов ресурсы были применены в Kubernetes-кластер, состоящий из master- и worker-ноды. Для изоляции ресурсов использовался отдельный namespace task3.

В процессе развертывания были созданы namespace, конфигурационные объекты, backend, frontend, PostgreSQL, постоянное хранилище и маршрутизация через Gateway API.

```
[maksimka@archlinux task3]$ kubectl apply -f k8s/
namespace/task3 unchanged
configmap/task3-app-config unchanged
deployment.apps/task3-backend unchanged
service/task3-backend unchanged
deployment.apps/task3-frontend unchanged
service/task3-frontend unchanged
secret/postgres-secret configured
configmap/postgres-config unchanged
service/postgres unchanged
service/postgres-db unchanged
statefulset.apps/postgres configured
gateway.gateway.networking.k8s.io/task3-gateway unchanged
httproute.gateway.networking.k8s.io/task3-route configured
persistentvolume/task3-postgres-pv configured
[maksimka@archlinux task3]$ kubectl get pods -n task3
NAME                                READY   STATUS    RESTARTS   AGE
postgres-0                          1/1     Running   6 (34m ago) 13d
task3-backend-67d8649fb-kqlrq       1/1     Running   0           25m
task3-frontend-75cb7d76b5-tdf2d     1/1     Running   1 (34m ago) 58m
task3-gateway-nginx-6486bf69db-p7css 1/1     Running   2 (34m ago) 70m
[maksimka@archlinux task3]$ kubectl get svc -n task3
NAME                                TYPE           CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
postgres                           ClusterIP      None          <none>         5432/TCP          13d
postgres-db                         ClusterIP      10.98.247.149 <none>         5432/TCP          13d
task3-backend                      ClusterIP      10.111.52.144 <none>         8080/TCP          13d
task3-frontend                     ClusterIP      10.100.250.35 <none>         80/TCP           13d
task3-gateway-nginx                 NodePort       10.103.45.209 <none>         80:32628/TCP     13d
[maksimka@archlinux task3]$ kubectl get pvc -n task3
NAME                                STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
postgres-data-postgres-0           Bound    task3-postgres-pv  1Gi        RWO            <default>      <unset>              13d
[maksimka@archlinux task3]$
```

После исправления сетевой конфигурации backend успешно подключился к PostgreSQL по имени сервиса postgres-db, а приложение стало корректно обрабатывать запросы.

Проверка проводилась несколькими способами:

- через curl к backend API;
- через открытие HTML-страницы в браузере;
- через добавление новой записи в PostgreSQL и повторную выборку списка данных.

Также была подтверждена работа приложения через Gateway API и NodePort.

```
[maksimka@archlinux ~]$ curl http://127.0.0.1:8080/api/people
{"count":8,"message":"People loaded from PostgreSQL","people":[{"age":23,"first_name":"Lev","id":1,"last_name":"Sidorov","middle_name":"Petrovich"},{"age":31,"first_name":"Anna","id":2,"last_name":"Smirnova","middle_name":"Igorevna"},{"age":27,"first_name":"Maksim","id":3,"last_name":"Orlov","middle_name":"Denisovich"},{"age":61,"first_name":"Maria","id":4,"last_name":"Orlov","middle_name":"Petrovich"},{"age":58,"first_name":"Ivan","id":5,"last_name":"Orlov","middle_name":"Sergeevna"},{"age":65,"first_name":"Olga","id":6,"last_name":"Morozov","middle_name":"Denisovich"},{"age":37,"first_name":"Anna","id":7,"last_name":"Morozov","middle_name":"Olegovich"},{"age":34,"first_name":"Maria","id":8,"last_name":"Orlov","middle_name":"Petrovich"}]}
[maksimka@archlinux ~]$
[maksimka@archlinux ~]$
[maksimka@archlinux ~]$
[maksimka@archlinux ~]$
[maksimka@archlinux ~]$ curl -X POST http://127.0.0.1:8080/api/people/random
{"message":"Random person added to PostgreSQL","person":{"age":23,"first_name":"Maria","id":9,"last_name":"Morozov","middle_name":"Ivanovna"}}
[maksimka@archlinux ~]$
```

## People loaded from PostgreSQL (9)

This page is rendered by the frontend and populated from PostgreSQL through the backend API.

Backend endpoint: `/api/people`

Use the button to create a new random record in PostgreSQL.

Add Random Person

|                         |              |
|-------------------------|--------------|
| Sidorov Lev Petrovich   | 23 years old |
| Smirnova Anna Igorevna  | 31 years old |
| Orlov Maksim Denisovich | 27 years old |
| Orlov Maria Petrovich   | 61 years old |
| Orlov Ivan Sergeevna    | 58 years old |
| Morozov Olga Denisovich | 65 years old |
| Morozov Anna Olegovich  | 37 years old |
| Orlov Maria Petrovich   | 34 years old |
| Morozov Maria Ivanovna  | 23 years old |

## Выводы

В ходе выполнения работы ранее подготовленное веб-приложение было успешно доработано и развернуто в Kubernetes-кластере с использованием PostgreSQL в качестве постоянного хранилища данных. Backend-приложение на Python было адаптировано для подключения к базе данных, чтения записей и добавления новых данных через REST API, а frontend обеспечивал отображение полученной информации в виде HTML-страницы.

В процессе выполнения задания были реализованы все основные требования:

- PostgreSQL был развернут в Kubernetes с использованием StatefulSet;
- для хранения данных были использованы PersistentVolume, PersistentVolumeClaim, Volume и VolumeMounts;
- параметры конфигурации приложения и базы данных были вынесены в ConfigMap и Secret;
- frontend и backend были развернуты как отдельные Deployment и Service;
- публикация приложения была выполнена через Gateway API с использованием Gateway и HTTPRoute вместо Ingress.

Таким образом, в рамках работы был получен практический опыт разработки и эксплуатации stateful-приложения в Kubernetes, настройки маршрутизации через Gateway API, организации постоянного хранения данных.